



Mahidol University
Wisdom of the Land



ITCS212 Web Programming

Introduction to MVC and Wrap up



Instructor: Wudhichart Sawangphol
Email: wudhichart.saw@mahidol.edu
Instructor: Jidapa Kraisangka
Email: jidapa.kra@mahidol.edu



- Students would be able to know concepts of the model-view-controller (MVC) architecture paradigm.
- Knowing frameworks that use to develop web application based on MVC.



- MVC
 - View + View Engine
 - Model (JSON and DB)
 - Controller
- Wrap Up and Beyond



Mahidol University
Wisdom of the Land



What is MVC?



```
// Import and initialization
const express=require('express');
const mysql = require('mysql2');
...
const app=express();
const router=express.Router();
const PORT = 3030;
app.use("/", router); //Register router1
...
// Routing
router.get('/', function (req, res){...});
router.get('/path1', function (req, res) { ... });
router.get('/path2', function (req, res) { ... });
...
// Database connection
var connection = mysql.createConnection({...});
connection.connect(function (err) {...});

// Web Service for connection to data source
router.get('/func1', function (req, res) { ... });
router.post('/ func2', function (req, res) { ... });
...

// Server running on the port
app.listen(PORT, function () {
    console.log("Server listening");
});
```

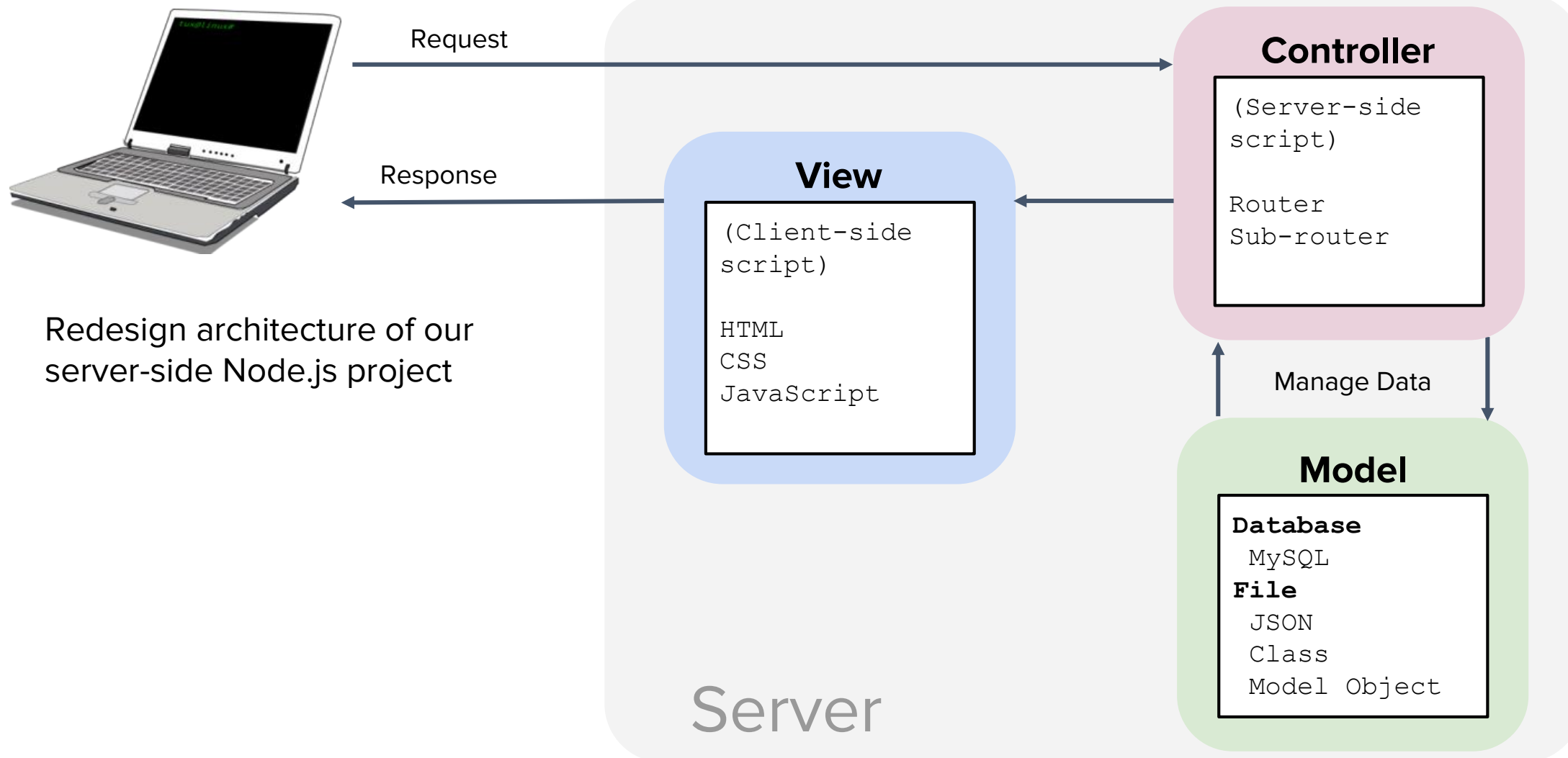
```
(root)
> node_modules
> public
.env
app.js
package-lock.json
package.json
<> login.html
<> homepage.html
<> somepage.html
...
```

Up to now

- Web server handles routing
- Web application (server-side codes) to retrieve the data from DB, respond to the request (dynamically)
- Every code is written in `app.js`



- **Model View Controller** (aka MVC) is a design pattern (architecture) created for developing specifically web applications.
- Dividing the component based on responsibility
 - **Model** - Anything related to data (i.e. JSON or DB)
 - **View** - Presentation of the data of the model to the user
 - **Controller** - Listens to all the event triggered in the view and performs an appropriate response back to the events.



Redesign architecture of our
server-side Node.js project



- Helps developer to focus on one specific element of the implementation at a time
- Scalable, clean, and robust.
- Efficient code reuse and parallel development



Welcome to ITCS212!!

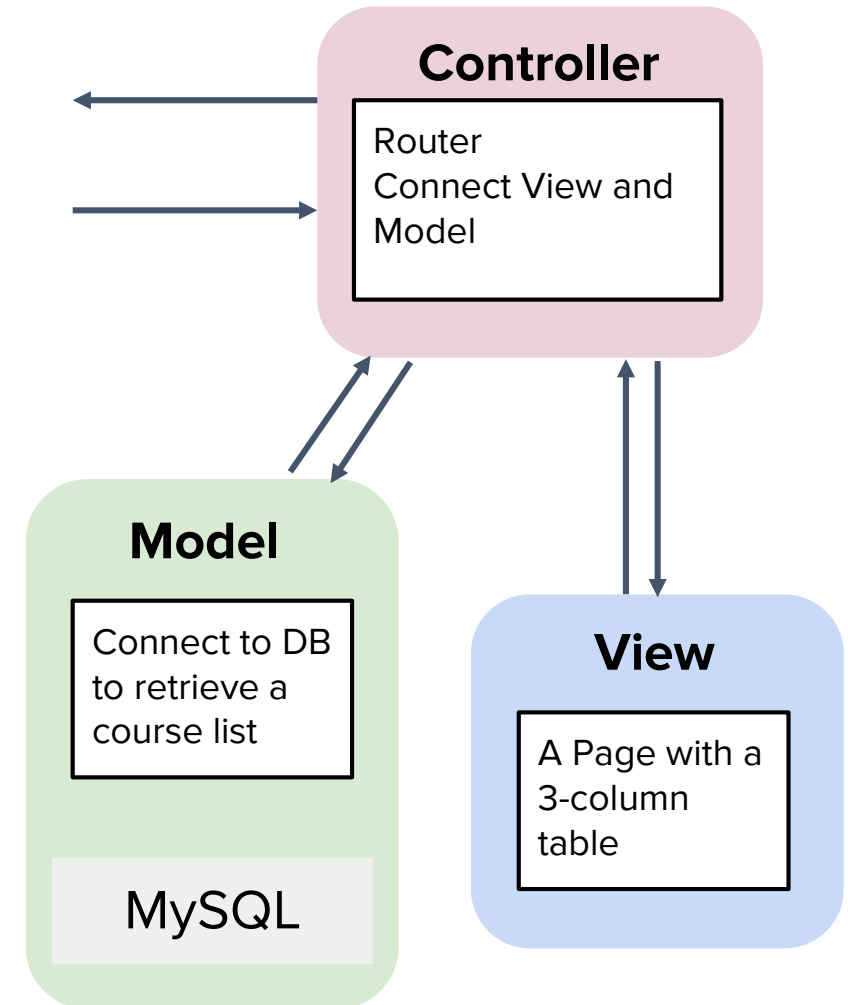
Web programming

Go to... [Home](#) [Course Outline](#)

Course Outline

Topic	Module #	Type
HTML/HTML5	1	Client-side
CSS+Bootstrap	2	Client-side
JavaScript	3	Client-side
Node.js	4	Server-side
Web Services	5	Server-side
REACT	6	Client-side

Thank you for the whole great semester!!!





Home

localhost:8081

Welcome to ITCS212!!

Web programming

Go to... Home Course Outline Student Info

Home Sweet Home

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Dapibus ultrices in iaculis nunc sed augue lacus. Quam nulla porttitor massa id neque aliquam. Ultrices mi tempus imperdiet nulla malesuada. Eros in cursus turpis massa tincidunt dui ut ornare lectus. Egestas sed sed risus pretium. Lorem dolor sed viverra ipsum. Gravidia rutrum quisque non tellus. Rutrum tellus pellentesque eu tincidunt tortor. Sed blandit libero volutpat sed cras ornare. Et netus et malesuada fames ac.

Ultrices eros in cursus turpis massa tincidunt dui ut ornare. Lacus sed viverra tellus in. Sollicitudin ac orci phasellus egestas. Purus in mollis nunc sed. Sollicitudin ac orci phasellus egestas tellus rutrum tellus pellentesque. Interdum consectetur libero id faucibus nisl tincidunt eget.

Thank you for the whole great semester!!!

Home

localhost:8081/course

Welcome to ITCS212!!

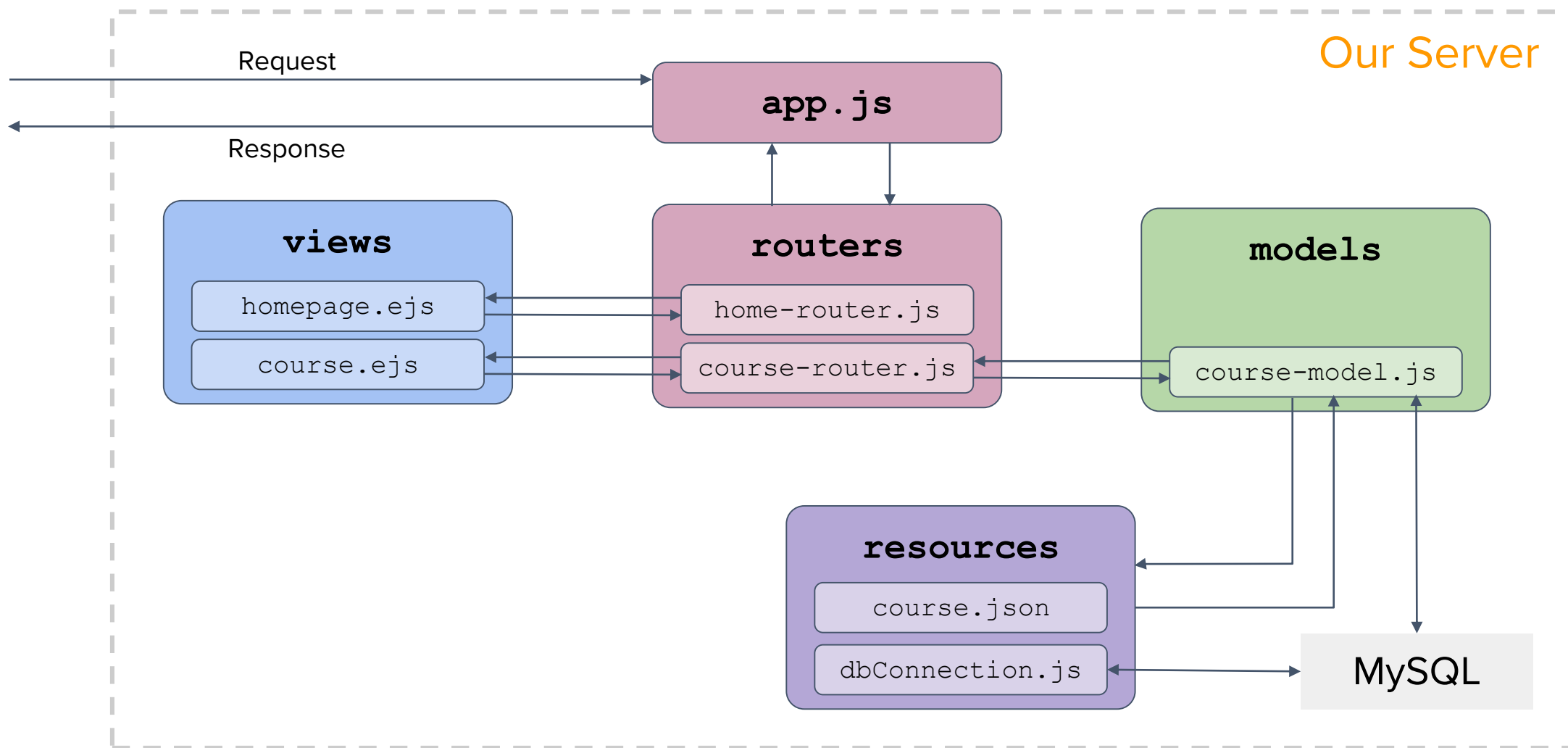
Web programming

Go to... Home Course Outline Student Info

Course Outline

Topic	Module #	Type
HTML/HTML5	1	Client-side
CSS+Bootstrap	2	Client-side
JavaScript	3	Client-side
Node.js	4	Server-side
Web Service	5	Server-side

Thank you for the whole great semester!!!

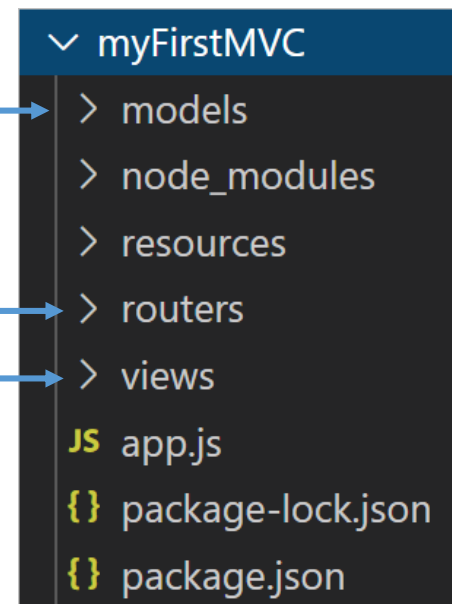




- Setting up your project based on model, view, controller
 - `npm init` (for initialization)
 - `npm install express --save` (for the express framework)
 - `npm install dotenv --save` (for setting env. variable)
 - `npm install nodemon --save`

- Organize your directory for easier mapping our files to MVC

- models: any files related to data
- views: any files related to presentation
- routers (controllers)





Mahidol University
Wisdom of the Land



V: Views + View Engine Template



- Part that interacts with users and only listen to the controller
- Output presentation of our web, e.g., content, tables, images etc.
- Using HTML and CSS.
** Faster develop the html page with the view engine template



- Easier to design an HTML page
- Provide static templates for rendering HTML
- At runtime, the engine **replaces variables in a template file** with actual values, and transforms the template into an HTML file sent to the client.
- Example **View engine** supported by Express:
 - **Pug**
 - **EJS**: Embedded JavaScript template engine.
 - **Handlebars**
 - **Mustache**



Welcome to ITCS212!!

Web programming

Go to... [Home](#) [Course Outline](#)

Course Outline

Topic	Module #	Type
HTML/HTML5	1	Client-side
CSS+Bootstrap	2	Client-side
JavaScript	3	Client-side
Node.js	4	Server-side
Web Services	5	Server-side
REACT	6	Client-side

Thank you for the whole great semester!!!

course.html

```
<table class="table">
  <tr>
    <th>Topic</th>
    <th>Module #</th>
    <th>Type</th>
  </tr>
  <tr>
    <td>HTML/HTML5</td>
    <td>1</td>
    <td>Client-side</td>
  </tr>
  <tr>
    <td>CSS+Bootstrap</td>
    <td>2</td>
    <td>Client-side</td>
  </tr>
  ...
  <tr>
    <td>REACT</td>
    <td>6</td>
    <td>Client-side</td>
  </tr>
</table>
```




Welcome to ITCS212!!

Web programming

Go to... [Home](#) [Course Outline](#)

Course Outline

Topic	Module #	Type
HTML/HTML5	1	Client-side
CSS+Bootstrap	2	Client-side
JavaScript	3	Client-side
Node.js	4	Server-side
Web Services	5	Server-side
REACT	6	Client-side

Thank you for the whole great semester!!!

course.ejs

```
<table class="table">
  <tr>
    <th>Topic</th>
    <th>Module #</th>
    <th>Type</th>
  </tr>
  <% for(let i=0; i<courses.length; i++) {
  %>
  <tr>
    <td><%= courses[i].name %></td>
    <td><%= courses[i].module %></td>
    <td><%= courses[i].type %></td>
  </tr>
  <% } %>
</table>
```



- EJS is a simple templating language that lets you generate HTML markup with plain JavaScript.
- Install EJS with npm (also add an extension in your VSCode)

```
npm install ejs --save
```
- No raw HTML file anymore, only **.ejs**
- Simple syntax with `<% %>`



app.js

```
const express = require('express');  
const dotenv = require('dotenv');  
const path = require('path');  
const app = express();  
dotenv.config();
```

```
/* Set view engine to be rendered */
```

```
app.set('views', path.join(__dirname, 'views'));  
app.set('view engine', 'ejs');
```

```
/* Running the server */
```

```
app.listen(process.env.PORT, function(){  
  console.log('Listening at Port ' + process.env.PORT);  
})
```

▼ views

> partials

<> course.ejs

<> index.ejs



Example: EJS files

homepage.ejs

```
<!DOCTYPE html><html language="en">
<head>
  <title>Home</title>
  <%- include('./partials/script'); %>
  <%- include('./partials/style'); %>
</head>
<body>
  <div class="container">
    <!-- Header --> <%- include('./partials/header'); %>
    <!-- Navbar --> <%- include('./partials/navbar'); %>
    <!-- Content -->
    <div class="row" style="margin-top:30px">
      <div class="col-md-8">
        <h1>Home Sweet Home</h1>
        <p>Paragraph 1...<p>
        <p>Paragraph 2...<p>
      </div>
    </div>
    <!-- Footer --> <%- include('./partials/footer'); %>
  </div>
</body>
</html>
```

partials/header.ejs

```
<header>
  <div class="mt-5 p-5 bg-light text-dark rounded text-center" style="margin-bottom:0">
    <h1>Welcome to ITCS212!!</h1>
    <p>Web programming</p>
  </div>
</header>
```

Note: Breaking down one EJS file into parts allows you to organize and **reuse** the codes

For the EJS syntax, use `<%-include('path'); %>` to include the EJS template file

- ✓ views
 - ✓ partials
 - <> footer.ejs
 - <> header.ejs
 - <> navbar.ejs
 - <> script.ejs
 - <> style.ejs
 - <> homepage.ejs



course.ejs

```
<body>
  <div class="container">
    <!-- Header --><%- include('./partials/header'); %>
    <!-- Navbar --><%- include('./partials/navbar'); %>
    <!-- Content -->
    <h1>Course Outline</h1>
    <table>
      <tr>
        <th>Topic</th>
        <th>Module #</th>
        <th>Type</th>
      </tr>
      <!-- Get Data from the Controller: courses -->
      <% for(let i=0; i<courses.length; i++) { %>
      <tr>
        <td><%= courses[i].name %></td>
        <td><%= courses[i].module %></td>
        <td><%= courses[i].type %></td>
      </tr>
      <% } %>
    </table>
    <!-- Footer --><%- include('./partials/footer'); %>
  </div>
```

Note: EJS is at the “**View**” level and it will communicate with the “Controller” only.

Assume we have the **courses** being sent from the “Controller”, we can bind/map each property to the template we want with JS

EJS Syntax

- Use `<% %>` for JS script
- Use `<%= %>` to output the value into the template



Home x + homepage.ejs

localhost:8081

Welcome to ITCS212!!

Web programming

Go to... Home Course Outline Student Info

Home Sweet Home

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Dapibus ultrices in iaculis nunc sed augue lacus. Quam nulla porttitor massa id neque aliquam. Ultrices mi tempus imperdiet nulla malesuada. Eros in cursus turpis massa tincidunt dui ut ornare lectus. Egestas sed sed risus pretium. Lorem dolor sed viverra ipsum. Gravida rutrum quisque non tellus. Rutrum tellus pellentesque eu tincidunt tortor. Sed blandit libero volutpat sed cras ornare. Et netus et malesuada fames ac.

Ultrices eros in cursus turpis massa tincidunt dui ut ornare. Lacus sed viverra tellus in. Sollicitudin ac orci phasellus egestas. Purus in mollis nunc sed. Sollicitudin ac orci phasellus egestas tellus rutrum tellus pellentesque. Interdum consectetur libero id faucibus nisl tincidunt eget.

Thank you for the whole great semester!!!

Home x + course.ejs

localhost:8081/course

Welcome to ITCS212!!

Web programming

Go to... Home Course Outline Student Info

Course Outline

Topic	Module #	Type
HTML/HTML5	1	Client-side
CSS+Bootstrap	2	Client-side
JavaScript	3	Client-side
Node.js	4	Server-side
Web Service	5	Server-side

Thank you for the whole great semester!!!



Mahidol University
Wisdom of the Land



M: Model



- Part of our application that will deal with the **database** or any **data-related** functionality
- If you need to define the structure of data, do it here :)
- Communicate only with the Controller (not to the view directly)
- Independent of user interface (views)
- Example:
 - Model with JSON
 - Model with Database



course.json

```
[
  {
    "name" : "HTML/HTML5",
    "module" : 1,
    "type" : "Client-side"
  },
  {
    "name" : "CSS+Bootstrap",
    "module" : 2,
    "type" : "Client-side"
  },
  ...
]
```

resources

data

{} course.json

models

JS course-model.js

Note: We create the model and export them to be used by the “Controller”

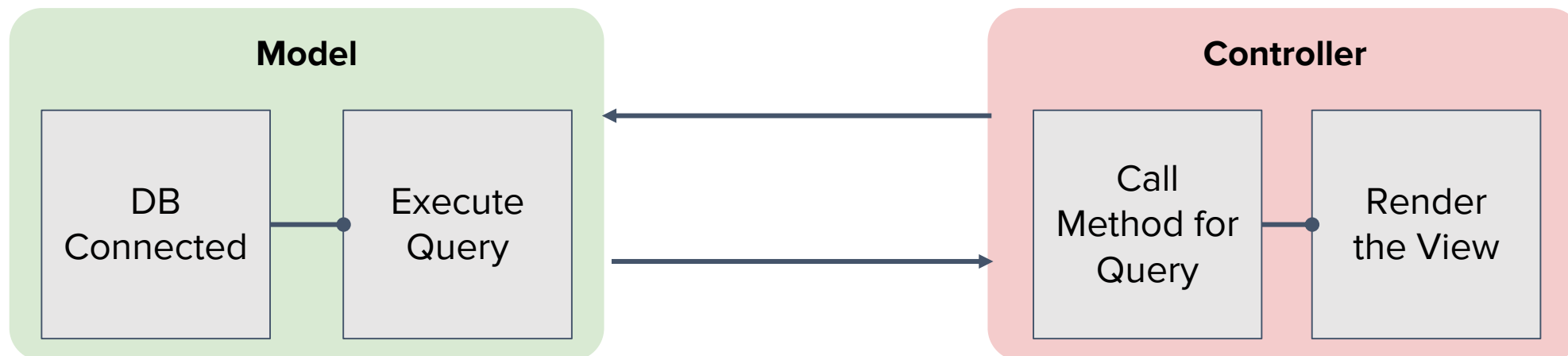
Do not forget to use **module.exports**, otherwise, no any other module can access to the model

course-model.js

```
/* Model: Connect to the Course JSON */
const courseJSON = require('../resources/data/course.json');
// You can create data model here if you want
module.exports = courseJSON;
```



- Like most I/O operations in Node.js, database access is asynchronous.
- However, some functions are needed to be completed before another





dbConnection.js

```
const mysql = require('promise-mysql');
const dbConfig = {
  host: process.env.MYSQL_HOST,
  user: process.env.MYSQL_USERNAME,
  password: process.env.MYSQL_PASSWORD,
  database: process.env.MYSQL_DATABASE
}
// Create a connection to DB
module.exports = async () => {
  try {
    let pool = await mysql.createPool(dbConfig);
    let connection = pool.getConnection();
    return connection;
  }
  catch (error) {
    throw error;
  }
}
```

A Promised-based version of MySQL allows us to use promise with DB operations

Defined in your **.env** file

Table:course (in MySQL DBMS)

name	module	type
HTML/HTML5	1	Client-side
CSS+Bootstrap	2	Client-side
JavaScript	3	Client-side
Node.js	4	Server-side
Web Service	5	Server-side

✓ resources

> data

JS dbConnection.js



course-model.js (cont.)

```
const dbConnection = require('../resources/dbConnection');  
class CourseMng{  
  async getAllCourses(){  
    // Get the connection  
    let connection = await dbConnection();  
    try {  
      let sql = "SELECT * FROM course";  
      let results = await connection.query(sql);  
      return results;  
    }  
    catch (error) {  
      throw error;  
    }  
  }  
  /* More SQL Methods here */  
}  
  
module.exports.CourseMng = CourseMng;
```

Note: We create **CourseMng** class for wrapping all DB method to be exported and used by the “Controller”

✓ models

JS course-model.js



Mahidol University
Wisdom of the Land



C: Controller



- Connects between models and views
- Do routing i.e. where to go to the models to get the data or do the action
- Render the views and send it to the users



- Define and use separate routers for each module
- Example**
home-router.js used for everything related to “Home” page

home-router.js

```
const express = require('express');  
const home = express.Router()  
  
home.get('/', function(req, res){  
  res.render('homepage');  
})  
  
module.exports = home;
```

▼ routers

JS course-router.js

JS home-router.js

When there is a GET request at the root, render **homepage.ejs**

Note: No need to specify the path since we already set up the view folder in **app.js**

Export the router to be called/used by **app.js**



course-router.js

```
const express = require('express');
const course = express.Router();

// Use this code if you want to use JSON
const courseList = require('../models/course-model').courseJSON;

course.get('/', function(req, res){
  console.log("From JSON");
  res.render('course', {'courses': courseList });
})

// Use this code if you want to connect with DB
const CourseMng = require('../models/course-model').CourseMng;
const courseMng = new CourseMng();

course.get('/', async function(req, res){
  console.log("From DB");
  const results = await courseMng.getAllCourses();
  res.render('course', {'courses': results });
})

module.exports = course;
```

models

JS course-model.js

views

> partials

<> course.ejs

<> homepage.ejs

Export the router to be called/used by
app.js

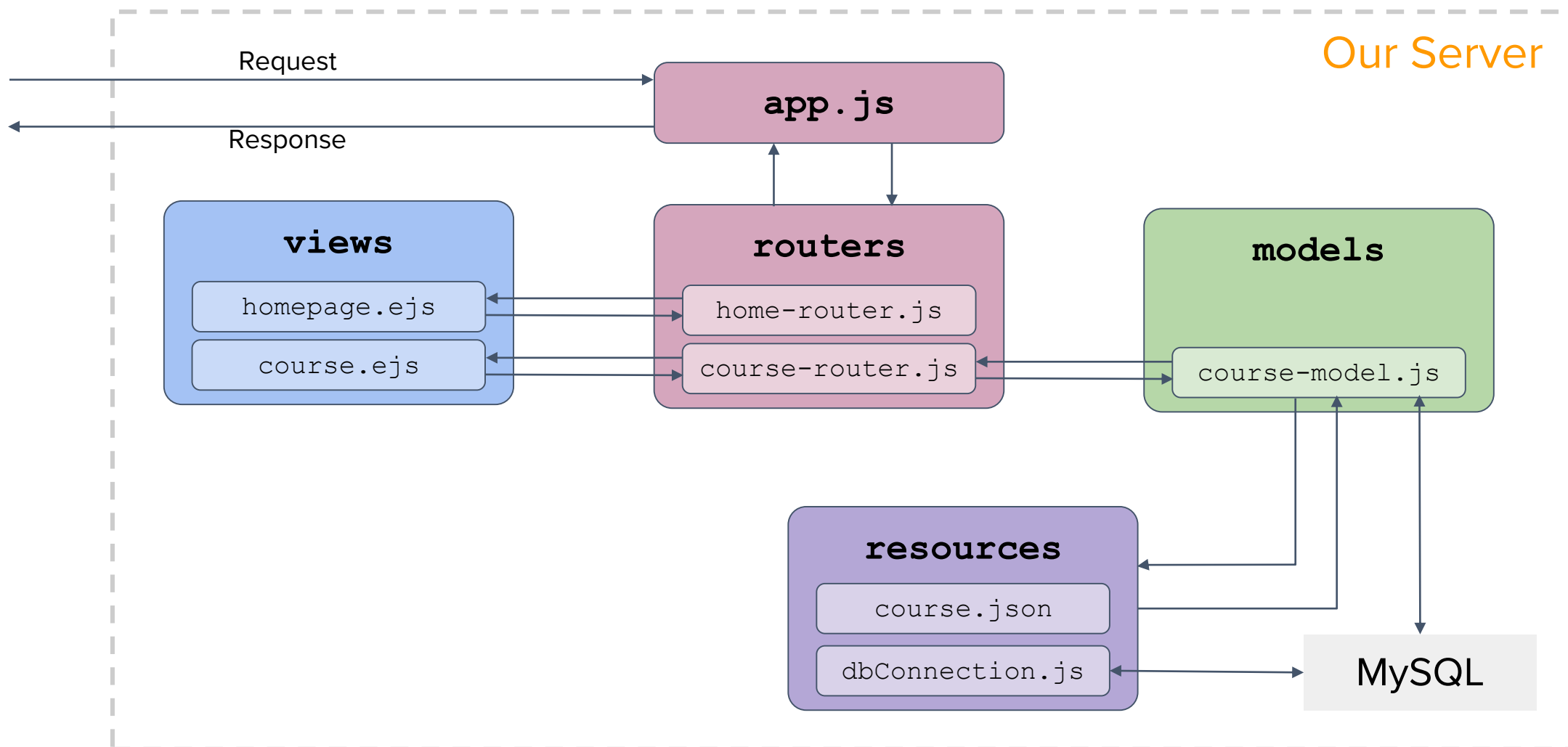


app.js (updated with Sub-Routers)

```
/* Import routers of each module */
const home = require('./routers/home-router');
const course = require('./routers/course-router');

/* Route to the page per request */
app.use('/', home); // http://localhost:8081
app.use('/course', course); // http://localhost:8081/course
```

Note: `app.use` manages the main routes and the **mini routers** handle the path for each module.





Mahidol University
Wisdom of the Land



Wrap Up and Beyond



Express 4.17.3

Fast, unopinionated,
minimalist web framework
for **Node.js**

Learn more about Node.js Framework choices:

- <https://www.simform.com/blog/best-nodejs-frameworks/>
- <https://hackr.io/blog/nodejs-frameworks>



- Other Web Development Framework choices:
 - **Django**: Model-View-Template framework that uses **Python** for web development.
 - **Rails**: a Model-View-Controller framework that uses **Ruby**
 - **Laravel**: a Model-View-Controller framework that uses **PHP**
 - **Spring**: a Model-View-Controller framework that uses **Java**
 - **ASP.NET Core**: an open source developer platform, created by Microsoft, for building many different types of applications, mainly **C#**



- Monolithic Architecture
- Microservices
- Serverless

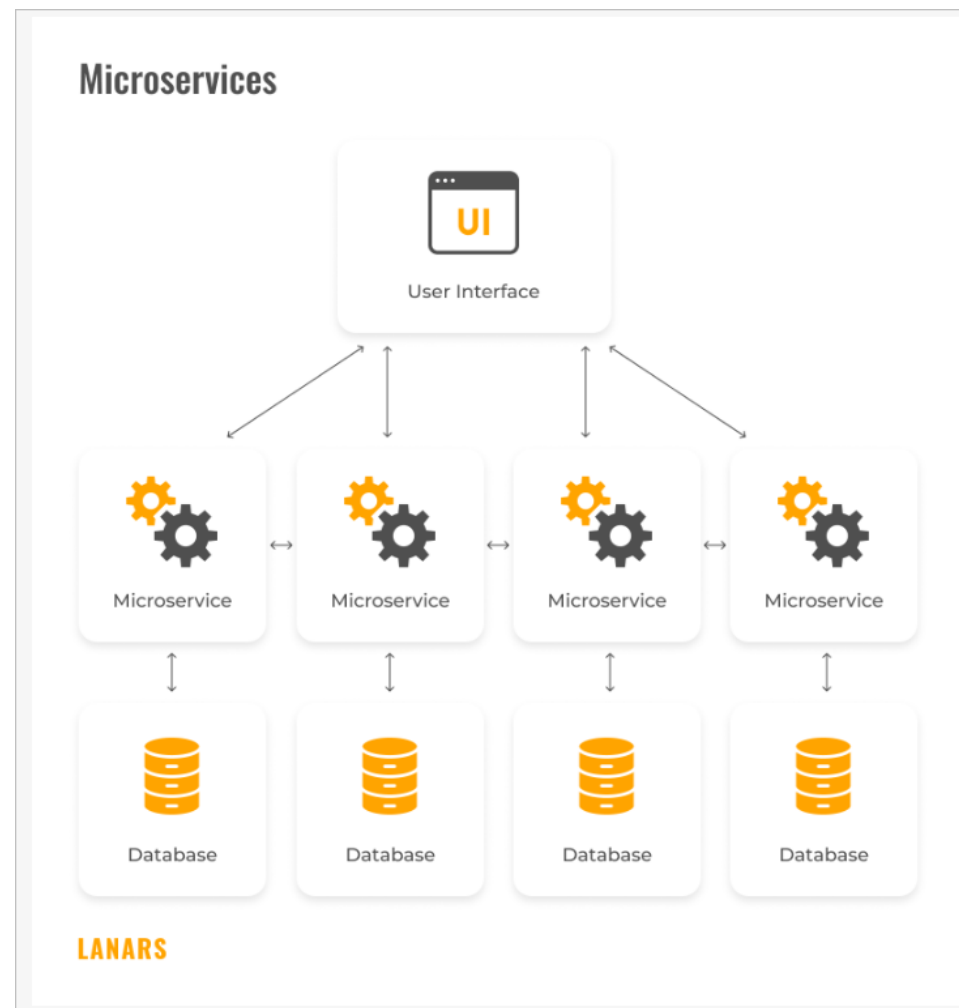


- Everything is managed and served from the same place
- Suitable for small-scale apps with simple functionality; low traffic.





- Everything is decomposing and decoupling it into smaller units.
- large-scale or complex products or web apps that you plan to scale; systems whose functionality you plan to upgrade regularly;



<https://www.divante.com/blog/monolithic-architecture-vs-microservices>

Image from LANARS: <https://lanars.com/blog/web-application-architecture-101>



- Allows developers to build and run services without having to manage the underlying infrastructure
- Offload some tasks to a cloud-service provider, e.g. AWS, Example: Function as a Service (FaaS): allows developers to focus on writing application code.

How Serverless Functions Work



Source: Datadog



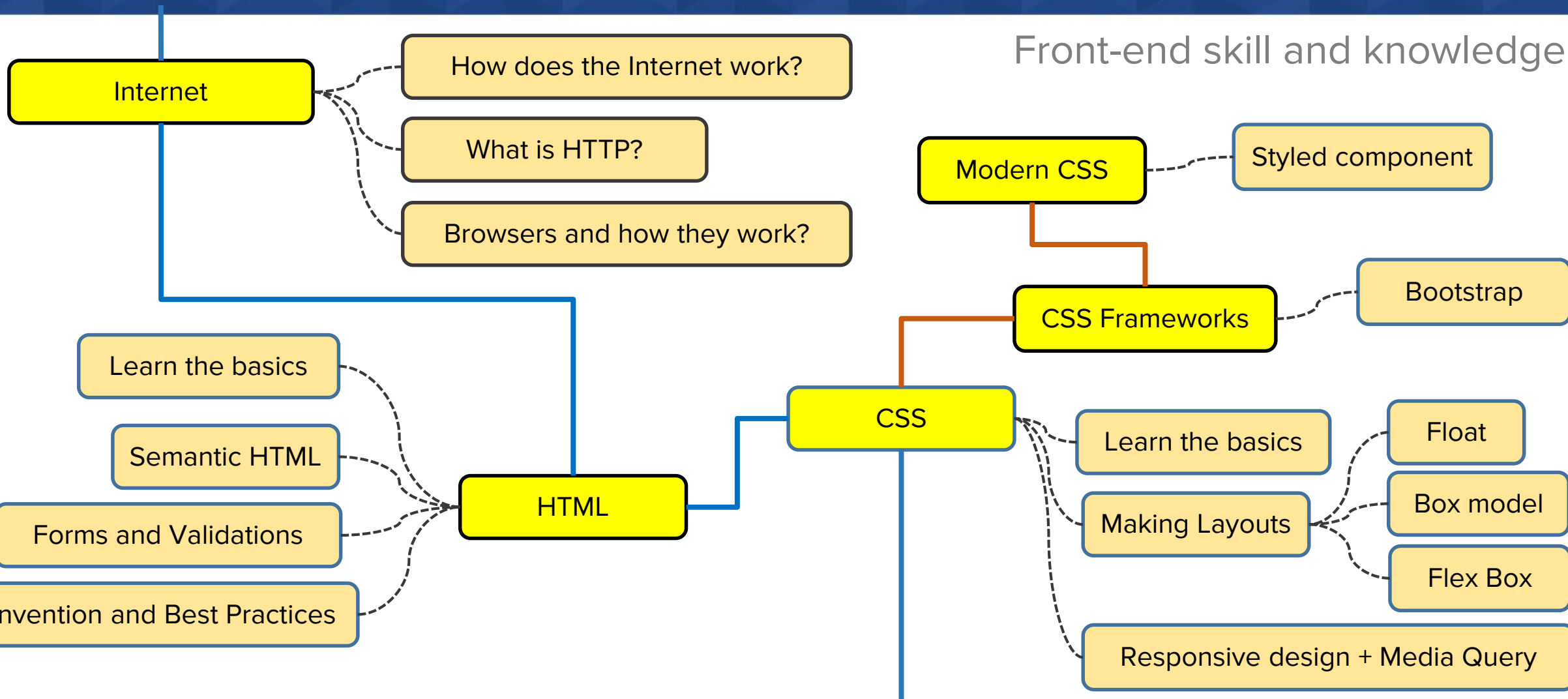
Mahidol University
Wisdom of the Land



Check List of ITC212

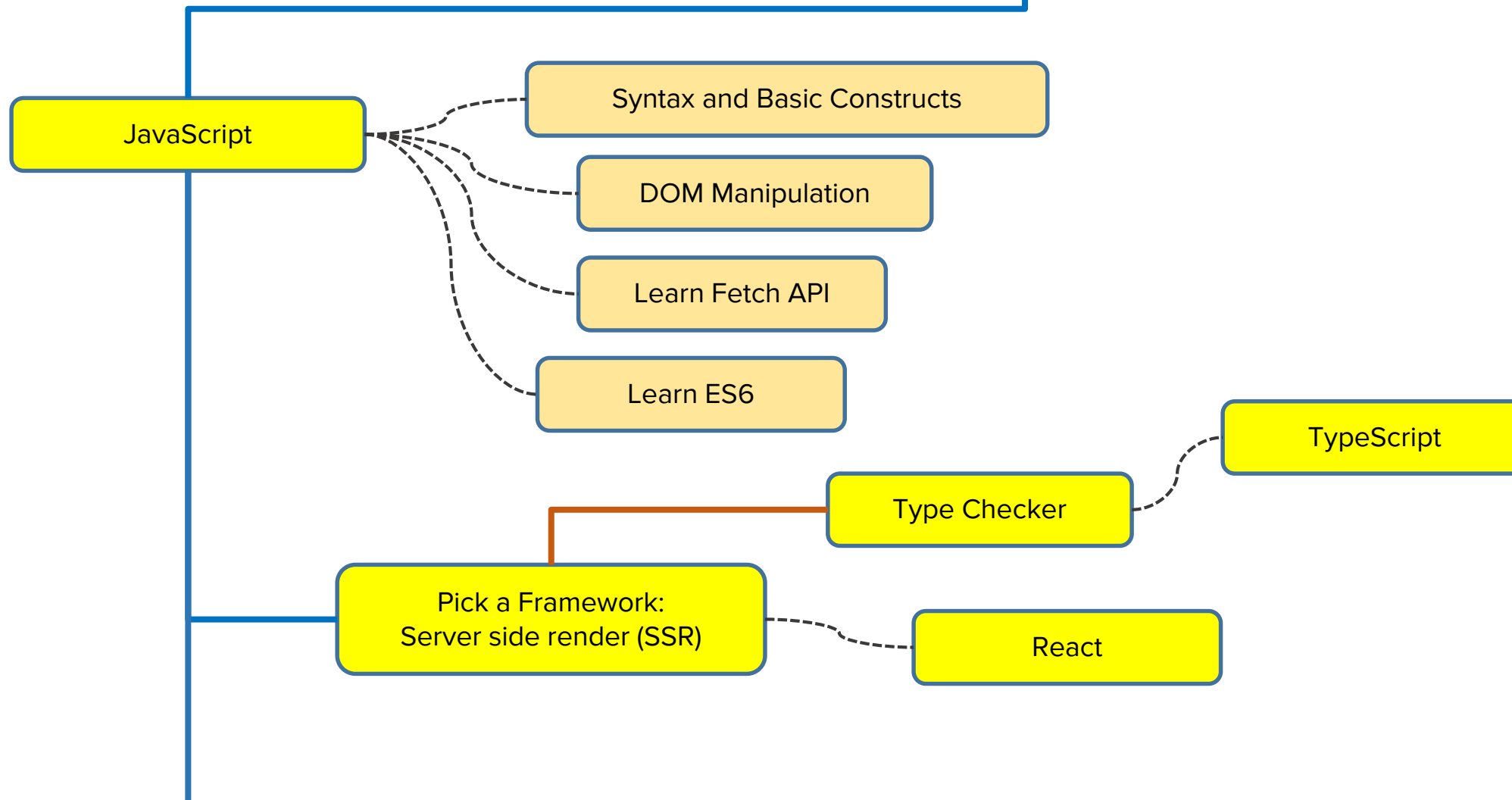


Front-end skill and knowledge





Front-end skill and knowledge





Back-end skill and knowledge

